Санкт-Петербургский Национальный Исследовательский Университет ИТМО
Факультет Программной Инженерии и Компьютерной Техники

Вариант №1234
Лабораторная работа №5
По дисциплине Программирование

Выполнил студент группы P3108:
Канищев Илья

Преподаватель:
Зайцева И. С.

Санкт-Петербург 2026 г.

Оглавление

Текст задания 3
Исходный код программы. 5
Диаграмма классов реализованной объектной модели..... 5
Вывод 5

Текст задания

Реализовать консольное приложение, которое реализует управление коллекцией объектов в интерактивном режиме. В коллекции необходимо хранить объекты класса `MusicBand`, описание которого приведено ниже.

Разработанная программа должна удовлетворять следующим требованиям:

- Класс, коллекцией экземпляров которого управляет программа, должен реализовывать сортировку по умолчанию.
- Все требования к полям класса (указанные в виде комментариев) должны быть выполнены.
- Для хранения необходимо использовать коллекцию типа `java.util.TreeMap`
- При запуске приложения коллекция должна автоматически заполняться значениями из файла.
- Имя файла должно передаваться программе с помощью: **аргумент командной строки**.
- Данные должны храниться в файле в формате `xml`
- Чтение данных из файла необходимо реализовать с помощью класса `java.util.Scanner`
- Запись данных в файл необходимо реализовать с помощью класса `java.io.BufferedOutputStream`
- Все классы в программе должны быть задокументированы в формате javadoc.
- Программа должна корректно работать с неправильными данными (ошибки пользовательского ввода, отсутсвие прав доступа к файлу и т.п.).

В интерактивном режиме программа должна поддерживать выполнение следующих команд:

- `help` : вывести справку по доступным командам
- `info` : вывести в стандартный поток вывода информацию о коллекции (тип, дата инициализации, количество элементов и т.д.)
- `show` : вывести в стандартный поток вывода все элементы коллекции в строковом представлении
- `insert null {element}` : добавить новый элемент с заданным ключом
- `update id {element}` : обновить значение элемента коллекции, id которого равен заданному
- `remove_key null` : удалить элемент из коллекции по его ключу
- `clear` : очистить коллекцию
- `save` : сохранить коллекцию в файл
- `execute_script file_name` : считать и исполнить скрипт из указанного файла. В скрипте содержатся команды в таком же виде, в котором их вводит пользователь в интерактивном режиме.
- `exit` : завершить программу (без сохранения в файл)
- `remove_greater {element}` : удалить из коллекции все элементы, превышающие заданный
- `replace_if_greater null {element}` : заменить значение по ключу, если новое значение больше старого
- `replace_if_lowe null {element}` : заменить значение по ключу, если новое значение меньше старого
- `average_of_albums_count` : вывести среднее значение поля albumsCount для всех элементов коллекции

- `filter_by_albums_count albumsCount` : вывести элементы, значение поля albumsCount которых равно заданному
- `print_field_descending_albums_count` : вывести значения поля albumsCount всех элементов в порядке убывания

Формат ввода команд:

- Все аргументы команды, являющиеся стандартными типами данных (примитивные типы, классы-оболочки, String, классы для хранения дат), должны вводиться в той же строке, что и имя команды.
- Все составные типы данных (объекты классов, хранящиеся в коллекции) должны вводиться по одному полю в строку.
- При вводе составных типов данных пользователю должно показываться приглашение к вводу, содержащее имя поля (например, "Введите дату рождения:")
- Если поле является enum'ом, то вводится имя одной из его констант (при этом список констант должен быть предварительно выведен).
- При некорректном пользовательском вводе (введена строка, не являющаяся именем константы в enum'е; введена строка вместо числа; введённое число не входит в указанные границы и т.п.) должно быть показано сообщение об ошибке и предложено повторить ввод поля.
- Для ввода значений null использовать пустую строку.
- Поля с комментарием "Значение этого поля должно генерироваться автоматически" не должны вводиться пользователем вручную при добавлении.

Описание хранимых в коллекции классов:

```
public class MusicBand {
    private long id; //Значение поля должно быть больше 0, Значение этого поля должно быть уникальным, Значение этого поля должно генерироваться автоматически
    private String name; //Поле не может быть null, Строка не может быть пустой
    private Coordinates coordinates; //Поле не может быть null
    private java.time.ZonedDateTime creationDate; //Поле не может быть null, Значение этого поля должно генерироваться автоматически
    private Integer numberOfParticipants; //Поле может быть null, Значение поля должно быть больше 0
    private Integer albumsCount; //Поле не может быть null, Значение поля должно быть больше 0
    private MusicGenre genre; //Поле может быть null
    private Person frontMan; //Поле может быть null
}
public class Coordinates {
    private double x;
    private Float y; //Максимальное значение поля: 968, Поле не может быть null
}
public class Person {
    private String name; //Поле не может быть null, Строка не может быть пустой
    private java.time.LocalDateTime birthday; //Поле может быть null
    private Integer height; //Поле может быть null, Значение поля должно быть больше 0
    private String passportID; //Длина строки не должна быть больше 28, Поле не может быть null
}

public enum MusicGenre {
    RAP,
    HIP_HOP,
    PSYCHEDELIC_CLOUD_RAP,
```

```
    SOUL,
    POST_ROCK;
}
```

Исходный код программы.

Код программы можно найти в репозитории:
https://github.com/kanishevilya/MusicBand-Laboratory-work/

Диаграмма классов реализованной объектной модели.

Диаграмма классов находится по ссылке:
https://github.com/kanishevilya/MusicBand-Laboratory-work/blob/main/UML.png

Вывод

В ходе выполнения лабораторной работы были изучены основные категории коллекций Java и методы сортировки элементов с использованием интерфейсов Comparable и Comparator. Были освоены параметризованные типы, wildcard-параметры, а также применены паттерны проектирования Command и Builder при разработке программы.